

# SMS Spam Detection ML Inference Service

Internship Project Report

Submitted: May 20, 2026

Tools: Python · scikit-learn · FastAPI · Joblib · Pytest

**97.94%**

Accuracy

**0.9535**

F1 Score

**5,574**

SMS Samples

**27**

Tests Passing

## 1. EXECUTIVE SUMMARY

This project delivers a complete, production-style machine learning inference service for SMS spam detection. Starting from a raw dataset of 5,574 SMS messages, the system was built through every stage of the ML engineering lifecycle: data exploration, text preprocessing, model training, REST API development, automated testing, and version-controlled deployment to GitHub.

The final system achieves 97.94% accuracy with a macro F1 score of 0.9535 on a held-out test set of 1,115 messages, with only 4 real messages wrongly flagged as spam. The trained model is served via a FastAPI REST endpoint with auto-generated interactive documentation and validated by 27 automated unit and integration tests.

## 2. PROBLEM STATEMENT

SMS spam wastes user attention, consumes mobile data, and facilitates phishing and fraud. Automated classification at the network or application layer is essential for user safety. The challenge is building a classifier that:

- Achieves high spam recall — missing spam is costly
- Maintains high ham precision — blocking real messages damages trust
- Runs fast enough for real-time inference on incoming messages

- Can be packaged and deployed as a standalone service

### 3. DATASET

The UCI SMS Spam Collection dataset was used, obtained from Kaggle. It contains 5,574 English SMS messages labelled as ham (legitimate) or spam.

| Property       | Value                                  |
|----------------|----------------------------------------|
| Source         | UCI ML Repository / Kaggle             |
| Total samples  | 5,574 messages                         |
| Ham (class 0)  | 4,827 (86.6%)                          |
| Spam (class 1) | 747 (13.4%)                            |
| Class ratio    | 6.5 : 1 (ham : spam)                   |
| Encoding       | latin-1 (non-ASCII characters present) |
| Format         | Tab-separated, quote-wrapped rows      |

The 6.5:1 class imbalance required stratified train/test splitting to ensure representative spam samples in both sets.

### 4. SYSTEM ARCHITECTURE

The project follows a modular, production-style architecture with clear separation of concerns across four layers:

| Module        | File              | Responsibility                                        |
|---------------|-------------------|-------------------------------------------------------|
| Data layer    | src/explore.py    | Dataset profiling, delimiter detection, null analysis |
| Preprocessing | src/preprocess.py | Text cleaning, TF-IDF vectorization, label encoding   |
| Training      | src/train.py      | Model training, evaluation, artifact serialization    |
| API layer     | api/main.py       | FastAPI server, /predict, /health endpoints           |
| Tests         | tests/test_api.py | 27 unit and integration tests via pytest              |

### 5. PREPROCESSING PIPELINE

Raw SMS text is transformed into numerical features through a four-step pipeline implemented in src/preprocess.py:

| Step           | Description                                                                                       |
|----------------|---------------------------------------------------------------------------------------------------|
| Text cleaning  | Lowercase conversion, URL removal, digit stripping, punctuation removal, whitespace normalization |
| Label encoding | Explicit map: ham → 0, spam → 1. Deterministic and human-readable.                                |

|                      |                                                                                          |
|----------------------|------------------------------------------------------------------------------------------|
| Stratified split     | 80/20 train/test split with stratify=y to maintain class ratio in both sets              |
| TF-IDF vectorization | max_features=5000, ngram_range=(1,2), sublinear_tf=True, min_df=2. Fitted on train only. |

Critical design decision: the TF-IDF vectorizer is fitted on training data only, then used to transform test data. Fitting on the full dataset before splitting constitutes data leakage and produces artificially inflated metrics.

## 6. MODEL SELECTION & TRAINING

Multinomial Naive Bayes was selected as the classifier with alpha=0.1 (Laplace smoothing).

**Justification for model choice:**

- Designed specifically for discrete word-count and TF-IDF features
- Trains in milliseconds — suitable for re-training on updated data
- Industry-standard baseline for text classification (used in early Gmail spam filter)
- Interpretable: word-level probability contributions can be inspected
- Appropriate for dataset size — neural approaches require far more data

The final model and fitted vectorizer are serialized to disk using Joblib:

```
models/spam_model.joblib - 157 KB
models/vectorizer.joblib - 183 KB
```

## 7. EVALUATION RESULTS

Evaluation was performed on 1,115 held-out test messages (20% stratified split). Accuracy alone is not reported as the primary metric due to class imbalance.

| Metric    | Ham (class 0) | Spam (class 1) | Overall        |
|-----------|---------------|----------------|----------------|
| Precision | 0.9806        | 0.9701         | —              |
| Recall    | 0.9959        | 0.8725         | —              |
| F1 Score  | 0.9882        | 0.9187         | 0.9535 (macro) |
| Support   | 966           | 149            | 1,115          |
| Accuracy  | —             | —              | 97.94%         |

**Confusion Matrix:**

|             | Predicted Ham | Predicted Spam |
|-------------|---------------|----------------|
| Actual Ham  | 962 (TN)      | 4 (FP)         |
| Actual Spam | 19 (FN)       | 130 (TP)       |

The 4 false positives (0.4% FP rate) represent real messages wrongly flagged as spam — an acceptably low rate. The 19 false negatives represent spam that reached the inbox. This tradeoff prioritizes user trust

over total spam elimination, which is the correct design decision for an SMS filter.

## 8. API DESIGN

The inference service is built with FastAPI and exposes three endpoints:

| Endpoint      | Method | Description                                  |
|---------------|--------|----------------------------------------------|
| GET /         | GET    | Service info and available endpoints         |
| GET /health   | GET    | Health check — returns model loaded status   |
| POST /predict | POST   | Classify a single SMS message                |
| GET /docs     | GET    | Auto-generated interactive API documentation |

### Sample /predict request and response:

```
POST /predict {"message": "FREE PRIZE!! Call 08001234 NOW!"}
{"message": "FREE PRIZE!! Call 08001234 NOW!", "cleaned": "free prize call now",
"label": "spam", "label_id": 1, "confidence": 0.9823, "is_spam": true}
```

The model is loaded once at server startup and reused across all requests — the standard pattern for low-latency ML inference services. Pydantic schemas enforce input validation automatically, returning HTTP 422 for malformed requests.

## 9. TESTING

A suite of 27 automated tests was written in tests/test\_api.py using pytest and FastAPI's TestClient. All 27 tests pass.

| Category              | Count | What is tested                                                                      |
|-----------------------|-------|-------------------------------------------------------------------------------------|
| Unit: clean_text()    | 8     | Lowercase, punctuation, URLs, digits, whitespace, edge cases                        |
| Integration: GET /    | 3     | Status 200, service name, docs path                                                 |
| Integration: /health  | 4     | Status 200, ok status, model loaded, version field                                  |
| Integration: /predict | 12    | Spam detection, ham detection, response schema, confidence range, validation errors |

## 10. SOFTWARE ENGINEERING PRACTICES

**Modular architecture** — Preprocessing, training, and API layers are fully independent modules

**DRY principle** — clean\_text() defined once in preprocess.py, imported by both train.py and api/main.py

**Reproducibility** — RANDOM\_STATE=42 ensures identical splits and results across runs

**Fail-fast design** — Server refuses to start if model artifacts are missing

**Input validation** — Pydantic schemas enforce type, length, and required fields automatically

**Conventional commits** — Git commit messages follow the feat:/fix:/docs: convention

**Dependency pinning** — requirements.txt generated via pip freeze for exact reproducibility

**Documentation** — Every function has a docstring with Args, Returns, and explanation

---

## 11. TECHNOLOGY STACK

| Technology   | Version | Purpose                                                          |
|--------------|---------|------------------------------------------------------------------|
| Python       | 3.10    | Primary language                                                 |
| scikit-learn | latest  | TF-IDF vectorization, Naive Bayes classifier, evaluation metrics |
| FastAPI      | latest  | REST API framework with auto-generated /docs                     |
| Uvicorn      | latest  | ASGI server for running FastAPI                                  |
| Joblib       | latest  | Model and vectorizer serialization                               |
| Pandas       | latest  | Dataset loading and manipulation                                 |
| Pytest       | latest  | Automated testing framework                                      |
| Git / GitHub | —       | Version control and project hosting                              |

---

## 12. CONCLUSION

This project successfully delivers a complete ML inference service for SMS spam detection, covering every stage from raw data to deployed API. The system achieves production-quality metrics (97.94% accuracy, 0.9535 macro F1) while demonstrating software engineering best practices including modular design, automated testing, and proper model serialization.

The most significant engineering challenge encountered was parsing an irregularly formatted dataset where pandas' standard CSV parser failed across multiple attempts. The solution — bypassing pandas entirely and using Python's built-in file reader — demonstrates the importance of debugging from first principles rather than assuming standard tools will always work.

Future improvements could include: Logistic Regression comparison, class-weighted training to improve spam recall beyond 87.2%, Docker containerization for cloud deployment, and a simple frontend UI for demonstration purposes.